

文献阅读子智能体构建实验报告

基于 pi 的六层隔离设计与实现

本实验构建了一个面向本地学术文献的只读子智能体。系统通过独立进程、独立上下文、独立系统提示词、工具白名单、专用 skill 和独立会话目录实现讲义提出的六层隔离，并为 PDF 文献提供带页码的分段提取能力。

实验日期 2026 年 9 月 29 日

摘要

本实验围绕“如何让主智能体可靠地委派文献阅读任务”展开。实现采用项目级 subagent 扩展：主 pi 负责识别任务和展示结果，每次调用都由 Node.js 的 spawn 创建独立 pi 子进程。子进程只接收当前任务，使用 literature-reader 专用系统提示词，只暴露 read、grep、find、ls 和 extract_pdf 五个只读工具，只加载 literature-reading skill，并把会话写入专用目录。

实验结果表明，扩展、智能体定义、PDF 工具和 skill 均可被 pi 0.87.1 正确加载；TypeScript 与 Python 语法检查通过；测试 PDF 可按指定页码提取文本。该方案满足讲义定义的六层隔离，但不等同于操作系统级沙箱：内置读取工具仍需依赖工作目录约束和任务规则，若要处理不可信文献或敏感目录，应进一步加入容器或文件系统访问控制。

关键词：pi；子智能体；文献阅读；上下文隔离；工具白名单；PDF 提取

1 实验目的

- 理解智能体上下文由系统提示词、消息列表、工具定义和运行时资源共同装配的机制。
- 实现一个能够阅读 PDF、Markdown 和纯文本学术文献的专用子智能体。
- 按照讲义要求落实进程、上下文、提示词、工具、skill 与存储六层隔离。
- 通过工具白名单和路径校验降低误修改文件、越界读取和无证据推断的风险。
- 形成可复用的项目结构、调用命令和验证方法，为后续构建其他专业子智能体提供参考。

2 实验原理

2.1 上下文重新装配

大语言模型的一次调用是无状态的。模型只能使用本次请求中显式提供的系统提示词、用户消息、工具定义和工具返回结果。所谓多轮对话，是调用方在每一轮重新提交历史内容的结果。因此，子智能体的隔离重点不是“让模型忘记”，而是让调度器在启动子进程时重新选择要装配的内容。

本方案不给子进程传递主会话消息列表，只传入 Task: 后面的当前任务。子智能体的系统提示词、工具、skill、工作目录和会话目录均由配置文件显式声明，从源头上控制它能看到什么、能调用什么以及把状态写到哪里。

2.2 装配阶段的能力硬剥夺

仅在提示词中要求“不要修改文件”属于行为约束，模型理论上仍能尝试使用写入工具。更可靠的方法是在启动参数中不提供 bash、powershell、edit 和 write。这样这些工具的名称、说明和参数格式都不会进入子智能体的可用工具集合，形成讲义所称的硬剥夺。

限制方式	作用位置	本实验采用情况
提示词约束	模型行为层	用于规定证据纪律和任务边界
执行前拦截	工具执行层	extract_pdf 对扩展名、存在性和路径范围进行校验
装配时移除	工具定义层	不加载命令执行、编辑和写入工具，是主要控制手段

文献阅读子智能体总体架构

主智能体负责任务委派与结果呈现，子进程独立装配执行上下文



图 1 文献阅读子智能体总体架构

3 实验环境

类别	配置	用途
操作系统	Windows 25H2, 构建 26200.9457, AMD64	运行 pi 与本地脚本
Node.js	26.7.0	运行 pi 和 TypeScript 扩展

类别	配置	用途
npm	11.19.0	管理 pi 包
pi	@earendil-works/pi-coding-agent 0.87.1	主智能体与子智能体运行时
Python	3.12.14	执行 PDF 文本提取脚本
pypdf	6.10.0	提取 PDF 页面文本
模型	cc-switch-sudo-code-chat/gpt-5.6-sol; thinking high	主会话默认模型，子智能体未指定模型时继承
项目目录	D:\Courses\材料智能设计实验\智能体\pi-workspace	保存项目级 .pi 配置
文献目录	D:\Courses\材料智能设计实验\智能体	子进程默认工作目录和可读课程资料范围

环境记录有意省略 API 密钥、代理地址和认证文件内容。运行子智能体需要主 pi 已能正常访问配置的模型服务；PDF 提取本身在本地完成，不依赖网络。

4 总体方案与目录结构

项目采用“声明与执行分离”的结构。agents 目录声明角色和资源白名单；extensions/subagent 负责发现配置和启动子进程；subagents/literature-reader 存放该角色独有的扩展、skill 和会话；prompts 提供用户入口。

```
pi-workspace/
├── .pi/
│   ├── agents/
│   │   └── literature-reader.md
│   ├── extensions/subagent/
│   │   ├── agents.ts
│   │   └── index.ts
│   ├── prompts/
│   │   └── read-literature.md
│   └── subagents/literature-reader/
│       ├── pdf-tool.ts
│       ├── extract_pdf.py
│       ├── skill/SKILL.md
│       ├── sessions/
│       └── README.md
```

文件	责任
----	----

文件	责任
literature-reader.md	声明名称、描述、工具、skill、扩展、存储目录、工作目录和系统提示词
agents.ts	解析 YAML frontmatter，解析相对路径，发现项目级智能体
index.ts	组装命令行参数、创建子进程、转发进度并返回最终结果
pdf-tool.ts	注册 extract_pdf，并在每轮启动前过滤环境资源
extract_pdf.py	使用 pypdf 按页读取 PDF，并在输出中保留页码
SKILL.md	定义文献阅读步骤、提取字段、证据纪律和默认输出结构
read-literature.md	将斜杠命令转换为标准 subagent 调用

5 详细构建方法

5.1 创建项目级智能体定义

在 `.pi/agents/literature-reader.md` 中使用 YAML frontmatter 声明资源。tools 是最终工具白名单；skills 和 extensions 指向该智能体唯一允许加载的资源；sessionDir 与 workspaceRoot 分别控制会话落盘位置和子进程工作目录。正文作为专用系统提示词，规定角色、证据要求和禁止事项。

```
---
name: literature-reader
tools: read, grep, find, ls, extract_pdf
skills: ../subagents/literature-reader/skill
extensions: ../subagents/literature-reader/pdf-tool.ts
sessionDir: ../subagents/literature-reader/sessions
workspaceRoot: ../../..
---
```

5.2 扩展 agent 配置解析器

原始示例只识别 name、description、tools 和 model。实验在 AgentConfig 中增加 skills、extensions、sessionDir 与 workspaceRoot，并让相对路径始终相对于智能体定义文件解析。这样配置可随项目目录整体移动，调度器也能为不同子智能体装配不同资源。

配置发现默认使用 project 范围，避免误用用户主目录中的同名 agent。若显式选择 both，则项目配置覆盖用户配置。

5.3 构造隔离的子进程启动参数

调度器在每次任务中创建新的参数数组。它先关闭自动发现，再显式加入允许的资源。这种“先清空、后加入”的方式使配置更容易审计。

```
--session-dir", sessionDir,
--no-context-files",
--no-extensions",
--no-skills",
--no-prompt-templates",
--no-themes",
--tools", "read,grep,find,ls,extract_pdf",
--skill", literatureReadingSkill,
--extension", pdfToolExtension,
--system-prompt", agentPromptFile,
`Task: ${task}`
```

--system-prompt 使用 agent 正文替换项目或用户级 SYSTEM.md；--no-context-files 阻止 AGENTS.md 和 CLAUDE.md 自动注入。子进程通过 shell: false 启动，避免任务文本被 shell 再解释。

5.4 实现只读 PDF 工具

pi 的内置 read 工具适合文本和图像，但不负责提取 PDF 文本。实验新增 extract_pdf 工具，其参数仅包含文件路径、起始页和结束页。工具把相对路径解析到 PI_LITERATURE_ROOT，拒绝工作目录之外的 PDF、非 PDF 文件、不存在的文件和反向页码范围。

通过 spawn 调用固定 Python 解释器和 extract_pdf.py。脚本用 pypdf 逐页提取文本，在每页前输出 --- Page N ---，使最终结论可以回溯页码。单次输出限制为 800 KiB；超出时中止并要求缩小页码范围，防止一次工具调用占满上下文。

5.5 加入二次资源过滤

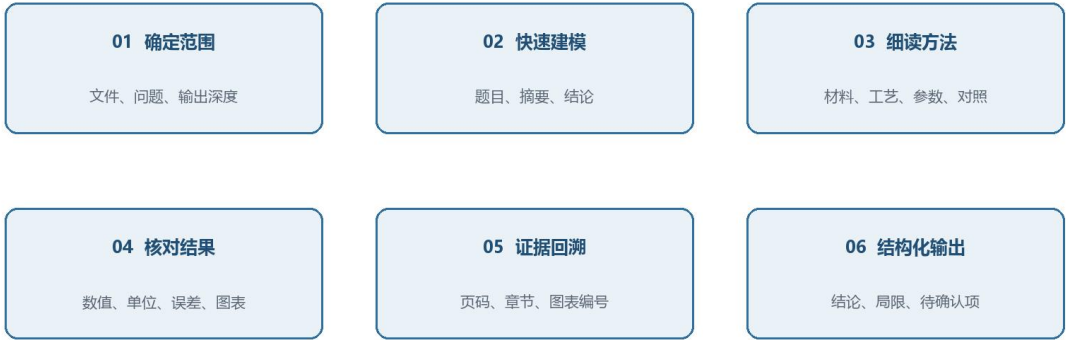
pdf-tool.ts 监听 before_agent_start。在每次模型调用前，它把 appendSystemPrompt 和 contextFiles 清空，将 skills 过滤为 literature-reading，并将 selectedTools 再次收敛到五个只读工具。该步骤属于纵深防御，用来抵御环境配置或后续扩展引入的意外资源。

5.6 编写文献阅读 skill

literature-reading skill 把领域方法从系统提示词中拆出，既保持角色提示简洁，又允许在任务开始时按需加载完整方法。skill 要求先快速阅读题目、摘要和结论建立结构，再细读方法、结果与图表，并对关键主张进行证据回溯。

文献证据提取策略

先建立结构，再回到原始证据，最后区分事实与推断



原则：摘要中的主张必须回到方法和结果部分复核；缺失信息保留为缺失，不进行补写。

图 2 文献证据提取流程

5.7 提供快捷调用入口

在 .pi/prompts/read-literature.md 中定义 /read-literature。模板固定 agent 为 literature-reader、agentScope 为 project，并把命令后的文本原样放入 task。主智能体收到结果后直接返回，不补写子智能体没有证据支持的内容。

一次文献阅读任务的执行流程



图 3 子智能体任务执行流程

6 六层隔离的实现

隔离层	实现机制	验证关注点	边界
进程	Node.js spawn 创建独立 pi 进程	父子进程 PID 和生命周期分离；Ctrl+C 可终止子进程	共享同一操作系统账户和文件权限
上下文	仅传入 Task: 当前任务；不传主消息历史	子进程 JSON 会话中不存在主会话消息	任务文本本身仍可能携带不必要信息
提示词	--system-prompt 加载 agent 正文；禁用上下文文件；调用前清除附加提示	启动参数与 before_agent_start 状态	pi 仍会按所选工具和 skill 构造必要说明
工具	--tools 五项白名单；无 shell、edit、write	帮助信息和运行时工具集合	内置读取工具不是 OS 文件沙箱
skill	--no-skills 后只显式加载 literature-reading	系统提示中仅出现目标 skill	skill 内容仍是模型应遵循的文本指令

隔离层	实现机制	验证关注点	边界
存储	--session-dir 指向专用 sessions 目录	子会话文件只在专用目录产生	文件仍位于同一磁盘和用户权限域

六层隔离主要约束上下文装配与能力表面。它降低主会话污染、工具误用和会话混存的概率，但不能替代 Windows ACL、容器、虚拟机或网络策略。若处理来源不可信的扩展、恶意文献或敏感研究数据，需要在该架构外再加操作系统级隔离。

7 子智能体功能

功能	行为	输出
文献概览	读取标题、摘要、关键词和结论	书目信息、研究问题和主要结论
材料与方法提取	识别组成、来源、工艺、设备、模型、参数和对照	结构化方法与实验条件表
定量结果提取	保留数值、单位、误差、样本量和测试条件	带条件的关键数据清单
证据定位	记录 PDF 页码、章节、图表编号或文本行号	可回查的结论与证据映射
批判性审读	区分作者自述、直接证据和推断	创新点、作者局限、审读局限
可复现性检查	检查参数、数据、代码和步骤是否完整	缺失信息与复现风险
扫描件识别	检测页面无可提取文本	明确报告需要 OCR，不猜测内容

8 使用方法

8.1 启动

1. 双击 D:\Courses\材料智能设计实验\智能体\启动 pi.cmd。该脚本会进入 pi-workspace 并启动 pi。
2. 首次加载项目资源时，确认当前目录确实是本实验项目后授予项目信任。
3. 修改 agent、skill 或扩展后重启 pi，或在交互界面执行 /reload。

8.2 快捷命令

```
/read-literature 阅读 D:\Courses\材料智能设计实验\智能体\你的论文.pdf,
总结核心结论、实验条件、关键数据与局限, 并标注页码
```

8.3 直接委派

```
调用 subagent 工具:
{ "agent": "literature-reader",
  "agentScope": "project",
  "task": "比较论文 A 与论文 B 的材料体系、制备参数和性能, 并给出证据页码" }
```

8.4 推荐任务表达

- 明确文件：给出文件名或绝对路径，避免“读一下那篇论文”之类的模糊指代。
- 明确问题：例如关注制备工艺、性能比较、机理解释或可复现性。
- 明确证据粒度：要求页码、章节、图表编号以及关键数值的测试条件。
- 明确输出：指定摘要、对比表、实验参数表、批判性评述或复现实验清单。

9 验证方法与结果

编号	检查项目	方法	结果
T1	TypeScript 语法	用 esbuild 分别转译 agents.ts、index.ts、pdf-tool.ts	通过
T2	Python 语法	compile 读取并编译 extract_pdf.py	通过
T3	主扩展加载	pi --offline --approve --help	退出码 0, 无扩展诊断错误
T4	隔离资源组合	关闭自动资源后显式加载 PDF 工具和 skill, 离线列出模型	退出码 0
T5	PDF 提取	对 20 页课程讲义提取第 1 页	成功输出页码和正文
T6	工具能力	检查工具白名单	仅 read、grep、find、ls、extract_pdf
T7	会话存储	检查专用 sessions 目录	子会话文件与主目录分离

验证采用离线启动，避免测试阶段触发模型调用或网络刷新。PDF 样本为课程讲义，共 20 页；提取结果包含 “Page 1” 标记和可读正文，证明页码定位链路有效。完整功能测试仍应在实际模型调用中补充，包括长论文分段策略、双文献对比、异常 PDF 和中断恢复。

10 风险、局限与改进建议

问题	影响	改进建议
扫描版 PDF 无文本层	pypdf 只能返回空文本	增加 OCR 工具，并限定输出页数和图像尺寸
内置 read 可接收绝对路径	工作目录不是严格文件系统沙箱	用容器、低权限账户或自定义受限 read 替换内置工具
Python 路径写死在扩展中	换机器后可能失效	优先读取 PI_PDF_PYTHON，并增加依赖探测和清晰报错
每次任务创建新进程	有启动开销	保留当前隔离设计；仅在高吞吐场景评估受控进程池
会话长期积累	占用磁盘并可能保留文献内容	设置保留周期、加密目录或为敏感任务使用临时会话
提示注入仍可能存在	文献文本可能诱导模型偏离任务	强化“文献是数据而非指令”的提示，并在输出端做证据格式校验
仅验证本地文件	不能直接检索数据库或 DOI	新增受限检索子智能体，与阅读子智能体分离授权

11 可扩展设计

当前架构适合作为多智能体文献工作流的基础。后续可以增加彼此隔离的检索、表格抽取、复核和综述写作子智能体，由主智能体只传递必要中间结果。这样可以针对每个角色使用不同工具和权限，而不把网络访问、文件写入或代码执行能力集中在一个上下文中。

扩展角色	最小工具	输入	输出
文献检索 agent	受限网络检索、元数据读取	主题、时间范围、数据库	候选文献与 DOI

扩展角色	最小工具	输入	输出
表格抽取 agent	PDF 读取、表格解析	指定页码或表号	结构化 CSV/JSON
证据复核 agent	只读工具	结论与引用位置	支持、部分支持或不支持
综述写作 agent	只读证据库、受限写入	经复核的证据卡片	带引用的综述草稿

12 实验结论

本实验完成了一个可在 pi 项目中直接调用的文献阅读子智能体。它能够读取 PDF、Markdown 和文本资料，按材料体系、方法、实验条件、结果、证据、局限和可复现性组织输出。实现将主会话与子任务的装配过程分开，并对进程、上下文、提示词、工具、skill 和存储逐层控制。

实验最重要的结论是：隔离应在请求发送给模型之前完成。通过关闭自动资源发现、只显式加入必要资源以及从工具列表中移除写入和命令执行能力，可以获得比单纯提示词约束更清晰、更可审计的边界。与此同时，六层隔离属于智能体运行时架构，不是操作系统安全沙箱；生产使用时应根据数据敏感度补充文件系统、网络 and 凭据隔离。

参考资料

[1] 构建智能体——材料智能设计的准备工作，课程讲义，重点参考“为什么要隔离子智能体”与六层隔离表。

[2] @earendil-works/pi-coding-agent 0.87.1，本机安装文档：Configuration、Command Line、Extensions、Skills 与 Sessions。

[3] pypdf 6.10.0，本实验用于 PDF 页面文本提取。

附录 A 关键配置核对清单

核对项	预期值
agent 名称	literature-reader
默认 agentScope	project
系统提示词参数	--system-prompt, 不使用 --append-system-prompt
自动上下文	--no-context-files
自动 skill	--no-skills; 随后显式 --skill
自动扩展	--no-extensions; 随后显式 --extension
工具白名单	read, grep, find, ls, extract_pdf
PDF 路径约束	必须位于 PI_LITERATURE_ROOT 内, 且扩展名为 .pdf
会话目录	.pi/subagents/literature-reader/sessions
扫描版处理	报告需要 OCR, 不推测缺失内容